Einführung in die Programmierung I

PD Dr. Andreas Paffenholz
Erik Jansen



TECHNISCHE UNIVERSITÄT DARMSTADT

WiSe 26/27
19.10.2026
Übungsblatt 1

Studienleistung

Die Studienleistung zu dieser Veranstaltung wird durch Abgabe von sinnvollen (mind. 20%) und eigenständig erarbeiteten **Lösungen zu 6 Blättern** und dem Erreichen von mindestens 50% der regulären Hausübungspunkte erbracht. Ihre Hausaufgaben werden von den Tutoren bewertet, und bei zwei bis drei Ihrer Abgaben werden Sie zu einem kurzen mündlichen Nachgespräch über Ihre Abgabe eingeladen. Die Punkte für diese Abgabe werden Ihnen dann erst nach einem erfolgreichen Gespräch gutgeschrieben.

Format der Abgabe

Die Abgabe der ersten Übung erfolgt in **Zweier-** oder **Dreiergruppen** durch das Hochladen Ihrer Dateien als **ein .tar-Archiv in Moodle**. Eine Anleitung zur Erstellung von .tar-Dateien finden Sie am Ende dieses Blattes. Ein Skript, dass Ihre Abgabe passend zusammenpackt, finden Sie auch in bei den Dateien, die wir Ihnen für die erste Übung zur Verfügung gestellt haben. Sie können das Skript in dem Ordner zur Abgabe durch

```
./loesung_zu_abgabe_umwandeln.sh
```

aufrufen.

Bitte geben **nur kompilierbaren Quellcode** ab, allerdings lediglich Ihren Quellcode und **keine kompilierten Programme** ab. Bitte ändern Sie die Dateinamen nicht ab und behalten das Schema mit `Aufgabe1.2.c`, `Aufgabe1.3.c`,... bei.

Im abgegebenen Archiv muss es **eine Datei `matrikelnummern.txt` enthalten** sein, in der für jedes Gruppenmitglied eine Zeile der Form

```
matrikelnummer, “Name”, “Vorname”, gitlab_user_id
```

enthalten ist. Eine Vorlage finden Sie auch bei den zur Verfügung gestellten Dateien.

Denken Sie auch daran, die Sie eine Erklärung zur Verwendung von KI und sonstigen Tools in der **Datei `erklaerung.txt` ausfüllen und mit abgeben** müssen. Eine Vorlage finden Sie in der Datei `erklaerung.txt` in den zur Verfügung gestellten Dateien. **Das Übernehmen von KI-Lösungen ist nicht zulässig** und wir von uns mit 0 Punkten bewertet.

Bitte vergessen Sie in keinem Fall die beiden .txt-Dateien `erklaerung.txt` und `matrikelnummern.txt`, da die Abgabe sonst nicht für alle Gruppenmitglieder angerechnet werden kann und mit 0 Punkten bewertet wird.

Aufgabe 1.1: Einführung

Absolvieren Sie die auf Moodle als .pdf veröffentlichte Einführung.

(Wenn Sie vertraut mit dem Umgang mit der Kommandozeile und dem Kompilieren von C-Programmen sind, können Sie diese Aufgabe überspringen. Diese Aufgabe soll nicht abgegeben werden)

Code-Templates

Für jede Aufgabe gibt es ein Code-Schnipsel mit Lücken, welchen Sie füllen sollen. Die notwendigen Dateien können Sie durch das Auführen des Kommandos

```
git clone https://gitlab.hefdi.de/tuda/einf_prog_i/2026/material/tutorials/uebung01.git
```

im Terminal herunterladen. Der notwendige Benutzername und das dazugehörige Passwort lauten:

Benutzername: `programmierung2026`

Passwort: `glpat-BbdmmWHW74cJv0dhV1Ex5m86MQp1OjFuZwk.01.0z0uhfxhl`

Code-Beispiel

Der folgende Quellcode ist ein Beispiel für ein Programm, welches eine Integer-Zahl x mittels `scanf("%d",&x)` einliest und alle Zahlen zwischen 0 und x mittles `printf("%d\n",i)` ausgibt.

```c
#include <stdio.h>

int main () {
        int x=0;

        printf("Insert an integer number\n");

        if (scanf("%d", &x) != 1) {
                printf("Error: Invalid input. Please enter an integer.\n");
                return 1;
        }

        for(int i = 0; i <= x; i = i + 1) {
                printf("%d\n", i);
        }
        return 0;
}
```

Das Schlüsselwort `for` beginnt hier eine for-Schleife. Diese wird wie folgt ausgeführt: Zuerst wird die Schleife mittels der Anweisung int `int i = 0` initialisiert. Anschließend wird solange der Inhalt des for-Blocks `printf("%d\n",i)` gefolgt von der Inkrementierung `i = i + 1` ausgeführt, bis die Forsetzungsbedingung `i <= x` nicht länger erfüllt ist.

Hinsweis: Nutzen Sie für das Einlesen von Eingaben aus der Kommandozeile die `scanf`-Funktion und für die Ausgabe in die Kommandozeile die `printf`-Funktion.

Aufgabe 1.2: Collatz-Folgen (1 Punkte)

Wir betrachten noch einmal das Programm zur Berechnung der Collatz-Folge aus der Vorlesung. Modifizieren Sie das Programm `Aufgabe1.2.c` so, dass

1. außerdem die Länge der Folge und
2. die Anzahl der ungeraden und geraden Folgenglieder ausgegeben wird.

Hinweis: Vergewissern Sie sich, dass die Summe der ungeraden und geraden Folgenglieder die Länge der Folgen ergibt.

Aufgabe 1.3: Rückgeldautomat (3 Punkte)

Vervollständigen Sie das Programm, welches für die Eingabe eines runden Eurobetrags/Preis ($n \in \mathbb{N}$) die kleinst mögliche Stückelung wiedergibt und die Anzahl an Münzen+Scheinen zählt.

Zusatz: Probieren Sie gerne auch andere Stückelungen (z.B. nur zwei oder drei Werte) und überlegen Sie sich inwiefern die Stückelungen reduziert werden können und einen Einfluss auf die auszahlbaren Werte haben (Tipp: Frobeniuszahlen).

Aufgabe 1.4: Primfaktorzerlegung (3 Punkte)

Eine Primfaktorzerlegung ist eine Darstellung von einer natürlichen Zahl n über das Produkt ihrer Primfaktoren, bspw. $28 = 2 \cdot 2 \cdot 7$. Falls die Zahl selbst eine Primzahl ist, besteht die Zerlegung nur aus der Primzahl selbst.

Vervollständigen Sie den gegebenen Programmcode so, dass die Primfaktorzerlegung über das Verfahren "Probedivision" berechnet wird.

Zunächst testen wir ob die eingegebene natürliche Zahl selbst schon ein triviale Primzahl ist. Anschließend beginnt die Probedivision mit dem kleinsten möglichen Primteiler $p = 2$ und überprüft, ob p ein Teiler der Eingabezahl n ist. Solange p ein Teiler ist, wird p als Primfaktor ausgegeben und n entsprechend verkleinert ($\frac{n}{p}$). Wenn p kein Teiler mehr von n ist, wird p erhöht. Hierbei ist es ausreichend neben 2 alle ungeraden Zahlen ≥ 2 zu betrachten. Der Prozess wird fortgesetzt, bis $p^2 > n$ gilt. Falls dann noch ein Rest $n > 1$ verbleibt, ist dieser selbst ein Primfaktor.

Aufgabe 1.5: Binärsystem (3 Punkte)

Zahlen lassen sich über verschiedene Zahlensystem darstellen. Üblicherweise werden Zahlen über das Dezimalsystem dargestellt, d.h. über die Darstellung in Zehnerpotenzen, bspw. $3756 = 3 \cdot 10^3 + 7 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$ oder allgemein $Z = \sum_{i=-n}^{m} z_i \cdot 10^i$ mit $z_m z_{m-1} \dots z_0 \, , z_{-1} z_{-2} \dots z_{-n} \;\; (m, n \in \mathbb{N}, z_i \in \{0, ..., 9\})$.

Schreiben Sie ein Programm, welches eine natürliche Zahl $Z \in \mathbb{N}$ aus dem Dezimalsystem in das Binärsystem überführt, d.h. über Zweierpotenzen dargestellt wird $Z = \sum_{i=0}^{m} z_i \cdot 2^i$ mit $z_m z_{m-1} \dots z_0 \, (m \in \mathbb{N}, z_i \in \{0, 1\})$

Aufgabe 1.6: Totales Querprodukt (2 Punkte)

Das Querprodukt $\mathrm{Q}(n)$ einer nicht-negativen ganzen Zahl $n \in \mathbb{N}_0$ ist das Produkt der Ziffern von n in der Dezimaldarstellung. Das totale Querprodukt $\mathrm{TQ}(n)$ erhält man durch wiederholtes Bilden des Querproduktes, bis das Ergebnis zwischen 0 und 9 liegt. Schreiben Sie ein Programm, welches $n \in \mathbb{N}_0$ als Eingabe nimmt und $\mathrm{TQ}(n)$ ausgibt und die als Zwischenschritt berechneten Querprodukte ebenfalls ausgibt.

Aufgabe 1.7: Bonus: Korrekt geklammerte Ausdrücke (3* Bonus-Punkte)

Wir betrachten Zeichenfolgen (oder Strings) s, welche aus den Zeichen `(` und `)` bestehen. In anderen Worten: $s \in \{$ `(` , `)` $\}$. Einen solchen String nennen wir *korrekt geklammert*, wenn er aus folgenden Regeln erzeugt werden kann:

1. Der leere String ist korrekt geklammert.
2. Ist s korrekt geklammert, so auch (s).
3. Sind s_1 und s_2 korrekt geklammert, so auch die Verkettung $s_1 s_2$.

Schreiben Sie ein Programm, welcher einen String s als Eingabe nimmt und ausgibt, ob dieser korrekt geklammert ist. Die Ausgabe in der Konsole sei 1 (`printf("1 \n");`), wenn er korrekt geklammert ist und 0 (`printf("0 \n");`), wenn nicht.

Abgabe der Übungen bis einschließlich 1.11.2026

Cheat Sheet: Terminal Befehle

Die wichtigsten Befehle für die Konsole haben wir hier zusammengestellt.

- `whoami` → (**WHO AM I**) Zeigt den Benutzernamen des aktuellen Benutzers an
- `pwd` → (**P**rint **W**orking **D**irectory) Zeigt den Namen des aktuellen Verzeichnisses an
- `ls` → (**Li**ste**S**ting) Zeigt den Inhalt des aktuellen Verzeichnisses an
- `cd` → (**C**hange **D**irectory) Wechselt das aktuelle Verzeichnis
- `mkdir` → (**M**a**K**e **DIR**ectory) Erzeugt ein neues Verzeichnis
- `cp` → (**C**o**P**y) Kopiert Dateien und Verzeichnisse
- `mv` → (**M**o**V**e) Verschiebt und benennt Dateien und Verzeichnisse um
- `rm` → (**R**e**M**ove) Löscht Dateien
- `rmdir` → (**R**e**M**ove **DIR**ectory) Löscht Verzeichnisse
- `gcc` → (**G**nu **C C**ompiler) Kompiliert .c Dateien
- `man` → (**MAN**ual) Hilfe für Unix Befehle und C Funktionen (Hinweis: `q` beendet das Anzeigeprogramm)

Sehr häufig geben Programme mit der Option `--help` oder `-h` eine Hilfe aus. Wenn Sie Hilfe zu einem Befehl und seinen Optionen benötigen, können Sie das sehr hilfreiche Tool `man` verwenden, z.B. `man ls` oder `man gcc`.

tar

Unter Linux ist das gleichnamige Programm `tar` zum Erstellen und Verarbeiten von Archiven zuständig.

Beispiele für Aufrufe von `tar`:

- `tar tzf Beispiel.tar.gz` – zeigt Inhalt der Datei an
- `tar xzf Beispiel.tar.gz` – extrahiert Datei
- `tar xf Beispiel.tar` – extrahiert nicht komprimiertes Tar-Archiv
- `tar czf Beispiel.tar.gz F1 F2 F3 F4` – Lege komprimiertes Tar-Archiv mit den Ordnern F1 bis F4 an